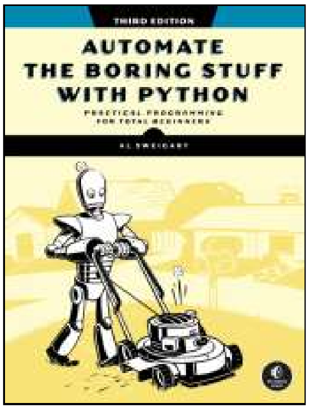
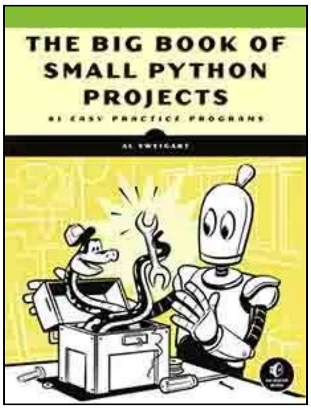
(fechar)

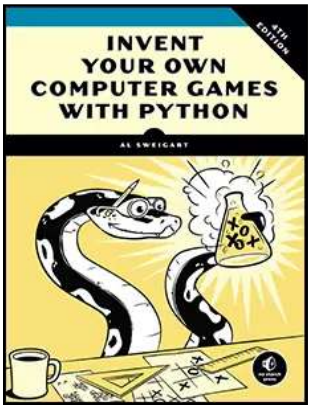
Confira outros livros de Al Sweigart, disponíveis **gratuitamente online (https://inventwithpython.com)** ou para compra: **...e outros livros também! (https://inventwithpython.com/)** Ou inscreva-se no **curso online em vídeo. (https://inventwithpython.com/automateudemy)** Você também pode **fazer uma doação para apoiar o autor diretamente. (https://inventwithpython.com/#donate)**

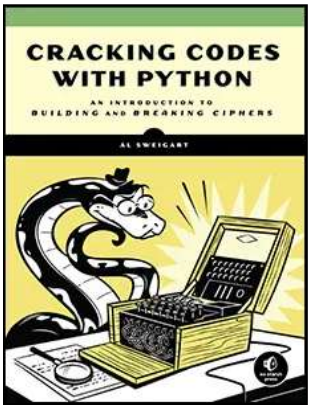


(https://nostarch.com/automate-boring-stuff-python-3rd-edition)



(https://nostarch.com/big-book-small-python-projects)





(https://nostarch.com/inventwithpython)
(https://nostarch.com/crackingcodes)

(https://inventwithpython.com/) (https://inventwithpython.com/automateudemy)
(https://inventwithpython.com/#donate)

INTRODUÇÃO

"Você acabou de fazer em duas horas o que nós três levamos dois dias para fazer." Meu colega de quarto da faculdade trabalhava em uma loja de eletrônicos no início dos anos 2000. De vez em quando, a loja recebia uma planilha com milhares de preços de produtos de outras lojas. Uma equipe de três funcionários imprimia a planilha em uma pilha grossa de papel e a dividia entre si. Para cada preço de produto, eles consultavam o preço em sua loja e anotavam todos os produtos que seus concorrentes vendiam por um preço menor. Geralmente, isso levava alguns dias.

"Sabe, eu poderia escrever um programa para fazer isso se vocês tivessem o arquivo original das impressões", disse meu colega de quarto quando os viu sentados no chão com papéis espalhados e empilhados por toda parte.

Após algumas horas, ele tinha um pequeno programa que lia o preço de um concorrente em um arquivo, encontrava o produto no banco de dados da loja e anotava o valor.se o concorrente era mais barato. Ele ainda era iniciante em programação, então passava a maior parte do tempo consultando a documentação em um livro de programação. O programa em si levava apenas alguns segundos para rodar. Meu colega de quarto e seus colegas de trabalho fizeram uma pausa para o almoço mais longa naquele dia.

Esse é o poder da programação de computadores. Um computador é como um canivete suíço, com ferramentas para inúmeras tarefas. Muitas pessoas passam horas clicando e digitando para realizar tarefas repetitivas, sem perceber que a máquina que estão usando poderia fazer o trabalho em segundos se recebessem as instruções corretas.

Para quem é este livro?

O software é essencial para muitas das ferramentas que usamos hoje: quase todo mundo usa redes sociais para se comunicar, praticamente todas as pessoas têm celulares com acesso à internet no bolso ou na bolsa, e a maioria dos trabalhos de escritório envolve a interação com um computador para realizar tarefas. Como resultado, a demanda por pessoas que sabem programar disparou. Inúmeros livros, tutoriais online e cursos intensivos para desenvolvedores prometem transformar iniciantes ambiciosos em engenheiros de software com salários de seis dígitos.

Este livro não é para essas pessoas. É para todos os outros.

Por si só, este livro não o transformará em um desenvolvedor de software profissional, assim como algumas aulas de guitarra não o transformarão em uma estrela do rock. Mas se você trabalha em um escritório, é administrador, acadêmico ou qualquer pessoa que use um computador para trabalho ou lazer, você aprenderá o básico de programação para automatizar tarefas simples como estas:

- Mover e renomear milhares de arquivos e organizá-los em pastas.
- Preenchimento de formulários online — sem necessidade de digitar
- Baixar arquivos ou copiar texto de um site sempre que ele for atualizado.
- Receba notificações personalizadas do seu computador no seu celular.
- Atualizar ou formatar planilhas do Excel
- Verificar seu e-mail e enviar respostas pré-escritas.
- Criar bases de dados e consultá-las para obter informações.
- Extração de texto de imagens e arquivos de áudio

Essas tarefas são simples, mas demoradas para os humanos, e muitas vezes são tão triviais ou específicas que não existe um software pronto para executá-las. No entanto, com um pouco de conhecimento de programação, você pode fazer com que seu computador execute essas tarefas para você.

Convenções de codificação usadas neste livro

Este livro não foi concebido como um manual de referência; é um guia para iniciantes. O estilo de codificação por vezes contraria as melhores práticas (por exemplo, alguns programas utilizam variáveis globais), mas essa compensação torna o código mais fácil de aprender. Conceitos de programação mais sofisticados — como **programação orientada a objetos (https://docs.python.org/3/tutorial/classes.html)** , **list comprehensions (https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions)** e **generators — (https://docs.python.org/3/tutorial/classes.html#generators)** não são abordados devido à simplicidade do livro .A complexidade que adicionam é um fator importante. Programadores experientes podem apontar maneiras de modificar o código deste livro para melhorar a eficiência, mas o foco principal deste livro é fazer com que os programas funcionem com o mínimo esforço da sua parte.

O que é programação?

Programas de televisão e filmes frequentemente mostram programadores digitando freneticamente sequências enigmáticas de 1s e 0s em telas brilhantes, mas a programação moderna não é tão misteriosa assim. *Programar* é escrever instruções para o computador executar em uma linguagem que ele possa entender. Essas instruções podem processar números, modificar texto, buscar informações em arquivos ou se comunicar com outros computadores pela internet.

Todos os programas usam instruções básicas como blocos de construção. Aqui estão algumas das mais comuns, em inglês:

“Faça isto; depois faça aquilo.”

“Se esta condição for verdadeira, execute esta ação; caso contrário, execute aquela ação.”

"Realize esta ação exatamente 27 vezes."

"Continue fazendo isso até que essa condição seja verdadeira."

Você pode combinar esses blocos de construção para implementar decisões mais complexas também. Por exemplo, aqui estão as instruções de programação, chamadas de *código-fonte*., para um programa simples escrito na linguagem de programação **Python (https://docs.python.org/3/)** . Começando do topo, o software Python executa linhas de código (algumas das quais são executadas somente *se* uma determinada condição for verdadeira, caso *contrário,* o Python executa alguma outra linha) até chegar ao final:

```
❶ password_file = open('SecretPasswordFile.txt')
❷ secret_password = password_file.read()
❸ print('Enter your password.')
typed_password = input()
❹ if typed_password == secret_password:
  ❺ print('Access granted')
  ❻ if secret_password == '12345':
      ❼ print('That password is one that an idiot puts on their luggage.')
  else:
    ❽ print('Access denied')
```

You might not know anything about programming, but you could probably make a reasonable guess at what the previous code does just by reading it. First, the file *SecretPasswordFile.txt* is opened ❶, and the secret password in it is read ❷. Then, the user is prompted to input a password (from the keyboard) ❸. These two passwords are compared ❹, and if they're the same, the program prints *Access granted* to the screen ❺. Next, the program checks whether the password is *12345* ❻ and hints that this choice might not be the best for a password ❼. If the passwords are not the same, the program prints *Access denied* to the screen ❽.

Programming is a creative task, as are painting, writing, knitting, and constructing LEGO castles. Like a blank canvas for painting, software has many constraints but endless possibilities. The difference between programming and other creative activities is that your computer comes with all the raw materials you need to program; you don't have to buy any additional canvas, paint, film, yarn, LEGO bricks, or electronic components. A decade-old laptop is more than powerful enough to write programs. Once you've written your program, you can copy it perfectly an infinite number of times. A knit sweater can be worn by only one person at a time, but a useful program can easily be shared online with the entire world.

What Is Python?

Python refers to both a programming language (with syntax rules for writing what is considered valid Python code) and the interpreter software that reads source code (written in the Python language) and performs its instructions. You can download Windows, macOS, and Linux versions of the Python interpreter for free at ***https://python.org (https://python.org)***.

There are several programming languages, each with their strengths and weaknesses. Debating which is best often leads to pointless arguments over matters of opinion. But in my opinion, Python is the best *first* language to learn if you are new to programming. Python has a gentle learning curve and readable syntax. It doesn't require learning dense concepts to do simple tasks. And if you want to go further into programming, learning a second language is easier when you first understand Python.

The name Python comes from the surreal British comedy group Monty Python, not from the snake. Python programmers are affectionately called Pythonistas, and both Monty Python and serpentine references usually pepper Python tutorials and documentation.

Common Myths About Programming

Programming has an intimidating reputation, and billion-dollar software companies are household names. Even the English word *code* has an association with secrecy and cryptic connotations. This leads many people to think that only a select few can program. But coding is a skill anyone can learn, and I'd like to address some of the more common myths directly.

Programmers Don't Need to Know Much Math

The most common anxiety I hear about learning to program is the notion that it requires a lot of math. Actually, most programming doesn't require math beyond basic arithmetic. Programming requires deduction and paying attention to detail more than mathematics. In fact, being good at programming isn't that different from being good at solving Sudoku puzzles.

To solve a Sudoku puzzle, you must fill in the numbers 1 through 9 for each row, each column, and each 3×3 interior square of the full 9×9 board. The puzzle provides you with some numbers to start, and you can find a solution by making deductions based on these numbers.

In the puzzle shown in Figure 1, a 5 appears in the first and second rows, so it can't show up in these rows again. Therefore, in the upper-right grid, it must appear in the third row. Because the last column also already has a 5 in it, the 5 can't go to the right of the 6, so it must go to the left of the 6. Solving one row, column, or square will provide more clues for solving the rest of the puzzle, and as you fill in one group of numbers 1 to 9 and then another, you'll soon solve the entire grid.

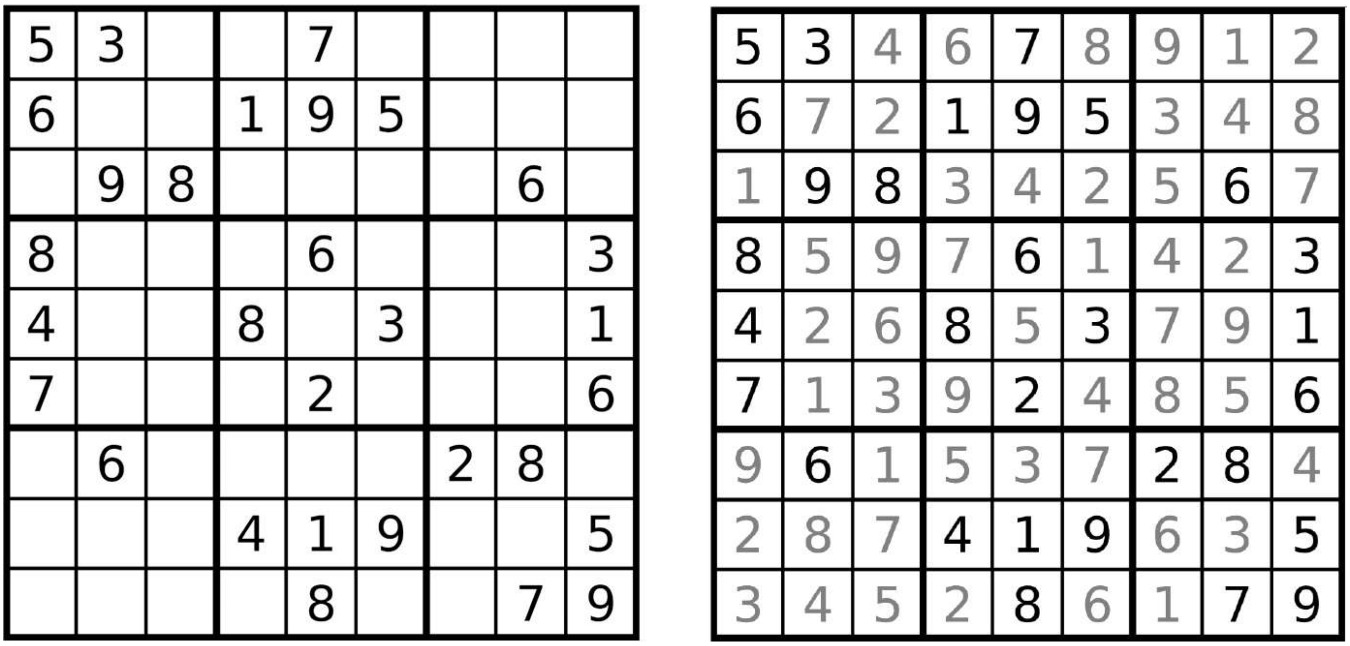


Figure 1: A new Sudoku puzzle (left) and its solution (right). Despite using numbers, Sudoku doesn't involve much math. (Images © Wikimedia Commons)

The fact that Sudoku involves numbers doesn't mean you have to be good at math to figure out the solution. The same is true of programming. Like solving a Sudoku puzzle, writing programs involves paying attention to details and breaking down a problem into individual steps. Similarly, when *debugging* programs (that is, finding and fixing errors), you'll patiently observe what the program is doing and find the cause of the bugs. And like all skills, the more you program, the better you'll become.

You Are Not Too Old to Learn Programming

The second most common anxiety I hear about learning to program is that people think they're too old to learn it. I read many internet comments from folks who think it's too late for them because they are already (gasp!) 23 years old. This is clearly not "too old" to learn to program; many people learn much later in life.

You don't need to have started as a child to become a capable programmer. But the image of programmers as whiz kids is a persistent one. Unfortunately, I contribute to this myth when I tell others that I was in grade school when I started programming.

However, programming is much easier to learn today than it was in the 1990s. Today, there are more books, better search engines, and many more online question-and-answer websites. On top of that, the programming languages themselves are far more user-friendly. For these reasons, *everything I learned about programming in the years between grade school and high school graduation could be learned today in about a dozen weekends*. My head start wasn't really much of a head start.

It's important to have a "growth mindset" about programming—in other words, understand that people develop programming skills through practice. They aren't just born as programmers, and being unskilled at programming now is not an indication that you can never become an expert.

AI Won't Replace Programmers

> In the 1990s, nanotechnology promised to change everything. New manufacturing processes and scientific innovation would revolutionize society, the thinking went. Carbon nanotubes, buckyballs, and diamonds the price of pencil lead, all assembled one atom at a time out of plentiful carbon by germ-size nanobots, would pave the way for materials 10 times stronger than steel at a fraction of the weight and cost. This would lead to space elevators, medical miracles, and home appliances that could create anything, just like the replicators on *Star Trek*! It would mean an end to economic scarcity, world hunger, and war. It would bring on a new age of enlightenment—as long as the nanobots didn't turn on their human creators in a tiny robot uprising. And the technology was only 10 years away!

Of course, the hype never happened. Real innovations certainly occurred at the nanoscale (the smartphone in your pocket uses a number of them). But the Star Trek replicators and other grandiose promises didn't arrive, and the excitement over nanotechnology deflated to more realistic proportions.

Let's talk about AI.

Personal computers changed everything. The internet changed everything. Social media changed everything. Smartphones changed everything. Cryptocurrency did not change everything, but it did reveal which of your cousins and co-workers were susceptible to get-rich-quick scams. Today, AI is the latest marvel to emerge from the tech industry. People use the term *AI* to mean everything from chess-playing computers to chatbots, so-called expert systems, and machine learning. In this book, I'll use the term *large language model (LLM)*, which is the conceptual category behind OpenAI's ChatGPT, Google's Gemini, Facebook's LLaMA, and other generative text systems.

LLMs have caused many breathless claims and questions. Is AI going to take all of our jobs? Is it still worth learning to code? Are the AIs alive, and can I survive the AI-robot uprising?

LLM technology is exciting, but to make it useful, we need to set realistic expectations so that we can avoid falling prey to sensationalist journalism and questionable "investment opportunities." I hope I can deflate the hype and give you a more realistic view of how LLMs can help you learn to code. Let's get some of these misconceptions out of the way right now:

- LLMs are not conscious or sentient.
- LLMs will not replace human software engineers.
- LLMs do not alleviate the need to learn programming.
- LLMs will not replace most human jobs (though this won't prevent your manager from thinking they can and laying you off anyway).
- LLMs are far from perfect, and are even *often* wrong.

Those who insist on these misconceptions get their information from science fiction movies and internet videos, not from experience with actual LLMs. I highly recommend Simon Willison, Python Software Foundation board member and co-creator of the Django Web Framework, for his writing about AI at ***https://simonwillison.net/about (https://simonwillison.net/about)***. You can watch his sober and illuminating PyCon 2024 keynote speech on LLMs at ***https://autbor.com/pycon2024keynote (https://autbor.com/pycon2024keynote)***.

How could LLMs help you as a programmer? What is obvious at this early stage is that learning to communicate with LLMs (so-called prompt engineering) is a skill, just like learning to effectively use a search engine is a skill. LLMs are not people, and you need to learn how to phrase your questions to get relevant and reliable answers. When LLMs confidently make up incorrect answers, we say that they are *hallucinating*. But this is just another way of anthropomorphizing an algorithm. LLMs don't think; they generate text. That is to say, LLMs are *always* hallucinating, even when their answers happen to be correct.

LLMs make large, obvious mistakes and simple, subtle mistakes, however. If you use them as a learning aid, you must vigilantly check everything the LLM tells you, big or small. It's entirely valid to choose to forgo learning with LLMs altogether. At this point, the effectiveness of using LLMs in education is unproven. We don't know for sure in what situations LLMs are useful as learning tools, or even if their benefits outweigh their costs.

It's no wonder that so many buy into the hype of LLMs. We carry devices in our pockets that scientists of the last century would consider supercomputers. They connect us to a global network of information (and misinformation). They can identify their position anywhere on Earth by listening to satellites in outer space. Software can already do seemingly magical things, but software isn't magic. And by learning to program, you'll get a far more grounded idea of what computers are capable of versus what is just hype.

About This Book

The first part of this book teaches you how to program in Python. The second part covers various software libraries for automating different kinds of tasks. I recommend reading the chapters of Part I in order, then skipping to the chapters in Part II that interest you. Here's a brief rundown of what you'll find in each chapter.

Part I: Programming Fundamentals

Chapter 1: Python Basics Covers expressions (https://docs.python.org/3/tutorial/introduction.html#numbers), the most basic type of Python instruction, and how to use the Python interactive shell software to experiment with code.

Chapter 2: if-else and Flow Control Explains how to make programs decide which instructions to execute so that your code can intelligently respond to different conditions.

Chapter 3: Loops Explains how to make programs repeat instructions a set number of times, or for as long as a certain condition holds.

Chapter 4: Functions Instructs you on how to define your own functions (https://docs.python.org/3/tutorial/controlflow.html#defining-functions) so that you can organize your code into more manageable chunks.

Chapter 5: Debugging Shows how to use Python's various bug-finding and bug-fixing tools (https://docs.python.org/3/library/debug.html).

Chapter 6: Lists Introduces the list data type (https://docs.python.org/3/tutorial/datastructures.html#more-on-lists) and explains how to organize data.

Chapter 7: Dictionaries and Structuring Data Introduces the dictionary data type (https://docs.python.org/3/tutorial/datastructures.html#dictionaries) and shows you more powerful ways to organize data.

Chapter 8: Strings and Text Editing Covers working with text data (called *strings* in Python).

Part II: Automating Tasks

Chapter 9: Text Pattern Matching with Regular Expressions Covers how Python can manipulate strings and search for text patterns with **regular expressions (https://docs.python.org/3/library/re.html)**.

Chapter 10: Reading and Writing Files Explains how your program can read the contents of text files and save information to files on your hard drive.

Chapter 11: Organizing Files Shows how Python can copy, move, rename, and delete large numbers of files much faster than a human user can. Also explains compressing and decompressing files.

Chapter 12: Designing and Deploying Command Line Programs Explains how you can package your Python programs to easily run them either on your own computer or on co-workers' computers.

Chapter 13: Web Scraping Shows how to write programs that can automatically download web pages and parse them for information. This is called *web scraping*.

Chapter 14: Excel Spreadsheets Covers programmatically manipulating Excel spreadsheets so that you don't have to read them. This is helpful when the number of documents you have to analyze is in the hundreds or thousands.

Chapter 15: Google Sheets Covers how to read and update Google Sheets, a popular web-based spreadsheet application, using Python.

Chapter 16: SQLite Databases Explains how to use relational databases with **SQLite (https://docs.python.org/3/library/sqlite3.html)**, the tiny but powerful open source database that comes with Python.

Chapter 17: PDF and Word Documents Covers programmatically reading Word and PDF documents.

Chapter 18: CSV, JSON, and XML Files Continues to explain how to programmatically manipulate documents, now discussing the data serialization formats CSV, JSON, and XML.

Chapter 19: Keeping Time, Scheduling Tasks, and Launching Programs Explains how Python programs handle **time and dates (https://docs.python.org/3/library/datetime.html)** and how to schedule your computer to perform tasks at certain times. Also shows how your Python programs can launch non-Python programs.

Chapter 20: Sending Email, Texts, and Push Notifications Explains how to write programs that can notify you via email or mobile communications, or send these messages to others.

Chapter 21: Making Graphs and Manipulating Images Explains how to programmatically manipulate images, such as JPEG or PNG files, and work with the Matplotlib graph-making library.

Chapter 22: Recognizing Text in Images Covers how to extract text from images and scanned documents for further processing with the PyTesseract package.

Chapter 23: Controlling the Keyboard and Mouse Explains how to programmatically control the mouse and keyboard to automate clicks and key presses.

Chapter 24: Text-to-Speech and Speech Recognition Engines Covers how to use advanced computer science packages to not only generate spoken audio from text, but also convert spoken audio to text.

You can download source code and other resources for the examples in this book at ***https://nostarch.com/automate-boring-stuff-python-3rd-edition (https://nostarch.com/automate-boring-stuff-python-3rd-edition)***. To see many of this book's programs in action, visit ***https://autbor.com/3 (https://autbor.com/3)***.

Downloading and Installing Python

You can download Python for Windows, macOS, and Ubuntu Linux for free at ***https://python.org/downloads (https://python.org/downloads)***.

The download page detects your operating system and recommends the download package for your computer. There are unofficial Python installers for Android and iOS mobile operating systems, but those are beyond the scope of this book. Windows, macOS, and Linux have their own installation options as well. Download the installer for your operating system and run the program to install the Python interpreter software.

New versions of Python or your operating system may change the steps needed to install Python. If you encounter difficulties, you can consult ***https://autbor.com/install/ (https://autbor.com/install/)*** for up-to-date instructions.

Downloading and Installing Mu

While the ***Python interpreter (https://docs.python.org/3/glossary.html#term-interpreter)*** is the software that runs your Python programs, the *Mu editor software* is where you'll enter your programs, much the way you enter text in a word processor. You can download Mu from ***https://codewith.mu (https://codewith.mu)***.

On Windows and macOS, download the installer for your operating system, then run it by double-clicking the installer file. If you are on macOS, running the installer opens a window where you must drag the Mu icon to the Applications folder icon to continue the installation. If you are on Ubuntu, you'll need to install Mu as a Python package. In that case, click the **Instructions** button in the Python Package section of the download page.

Starting Mu

Once it's installed, you can start Mu:

- On Windows, click the Start icon in the lower-left corner of your screen, enter **Mu Editor** in the search box, and select it.
- On macOS, open the Finder window, click **Applications**, and then click **mu-editor**. You can also run **Mu Editor** from Spotlight.
- On Ubuntu, select **Applications▯Accessories▯Terminal** and then enter python3 –m mu.

The first time Mu runs, a Select Mode window will appear with options for Adafruit CircuitPython, BBC micro:bit, Pygame Zero, Python 3, and others. Select **Python 3**. You can always change the mode later by clicking the **Mode** button at the top of the editor window.

Starting IDLE

This book uses Mu as an editor and interactive shell. However, you can use any number of editors for writing Python code. The ***interactive development environment (IDLE) (https://docs.python.org/3/library/idle.html)*** software installs along with Python, and it can serve as a second editor if for some reason you can't get Mu installed or working. Let's start IDLE now (assuming Python 3.13 is the version you installed):

- On Windows, click the Start icon in the lower-left corner of your screen, enter **IDLE** in the search box, and select **IDLE (Python 3.13 64-bit)**.
- On macOS, open the Finder window, click **Applications**, click **Python 3.13**, and then click the IDLE icon. You can also run IDLE from Spotlight.
- On Ubuntu, select **Applications□Accessories□Terminal** and then enter idle. (You may also be able to click **Show Apps** at the bottom of the Ubuntu sidebar and then click **IDLE**.)

The Interactive Shell

When you run Mu, the window that appears is called the *file editor* window. You can open the *interactive shell* by clicking the REPL button. A **shell (https://docs.python.org/3/tutorial/interpreter.html)** is a program that lets you enter instructions into the computer, much like the Terminal or Command Prompt on macOS and Windows, respectively. Python's interpreter software will immediately run any instructions you enter into the Python interactive shell.

In Mu, the interactive shell is a pane in the lower half of the window with something like the following text:

```
Jupyter QtConsole
Python 3
Type 'copyright', 'credits' or 'license' for more information
IPython -- An enhanced Interactive Python. Type '?' for help.

In [1]:
```

If you run IDLE, the interactive shell is the window that first appears. It should be mostly blank except for text that looks something like this:

```
>>>
```

`In [1]:` and `>>>` are called *prompts*. The examples in this book will use the `>>>` prompt to represent the interactive shell, since it's more common. If you run Python from the Terminal or Command Prompt, they'll use the `>>>` prompt as well. The `In [1]:` prompt was invented by Jupyter Notebook, another popular Python editor.

For example, enter the following into the interactive shell next to the prompt:

```
>>> print('Hello, world!')
```

After you write the line and press ENTER, the interactive shell should display this in response:

```
>>> print('Hello, world!')
Hello, world!
```

You've just given the computer an instruction, and it did what you told it to do!

How to Find Help

Programmers tend to learn by searching the internet for answers to their questions. This is quite different from the way many people are accustomed to learning—through an in-person teacher who lectures and can answer questions. What's great about using the internet as a schoolroom is that there are whole communities of folks who can help you solve your problems. Indeed, your questions have probably already been answered, and the answers are waiting online for you to find them. If you encounter an error message or have trouble making your code work, you won't be the first person to have your problem, and finding a solution is easier than you might think.

For example, let's cause an error on purpose: enter '42' + 3 into the interactive shell. You don't need to know what this instruction means right now, but the result should look like this:

```
>>> '42' + 3
❶ Traceback (most recent call last):
  File "<pyshell#0>", line 1, in <module>
    '42' + 3
❷ TypeError: can only concatenate str (not "int") to str
>>>
```

The error message ❷ appears because Python couldn't understand your instruction. The traceback part ❶ of the error message shows the specific instruction and line number that Python had trouble with. If you're not sure what to make of a particular error message, search for it online. Enter **"TypeError: can only concatenate str (not "int") to str"** (including the quotation marks) into your favorite search engine, and you should see tons of links explaining what the error message means and what causes it, as shown in Figure 2.

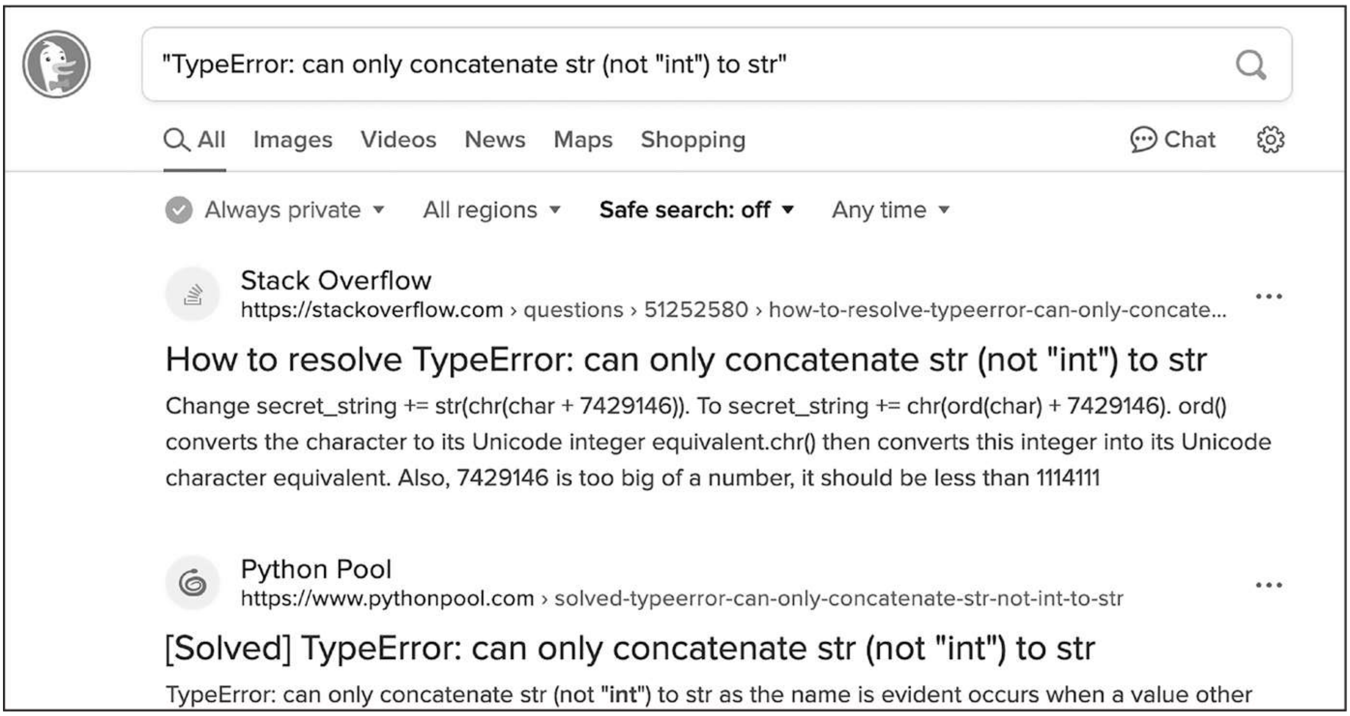


Figure 2: The Google results for an error message can be very helpful.

You'll often find that someone else had the same question as you and that some other helpful person has already answered it. No one person can know everything about programming, so an everyday part of any software developer's job is looking up answers to technical questions.

Asking Smart Programming Questions

If you can't find the answer by searching online, try asking people in a web forum such as Stack Overflow ([*https://stackoverflow.com*](https://stackoverflow.com)

(https://stackoverflow.com)) or the "learn programming" subreddit at ***https://reddit.com/r/learnprogramming (https://reddit.com/r/learnprogramming)***. But keep in mind there are smart ways to ask programming questions that help others help you. To begin with, be sure to read the Frequently Asked Questions sections at these websites about the proper way to post questions.

When asking programming questions, remember to do the following:

- Explain what you are trying to do, not just what you did. This lets your helper know if you are on the wrong track.
- Specify the point at which the error happens. Does it occur at the very start of the program or only after you do a certain action?
- Copy and paste the *entire* error message and your code to ***https://pastebin.com (https://pastebin.com)*** or ***https://gist.github.com (https://gist.github.com)***. These websites make it easy to share large amounts of code with people online, without losing any text formatting. You can then put the URL of the posted code in your email or forum post. For example, here are the locations of some pieces of code I've posted: ***https://pastebin.com/2k3LqDsd (https://pastebin.com/2k3LqDsd)*** and ***https://gist.github.com/asweigart/6912168 (https://gist.github.com/asweigart/6912168)***.
- Explain what you've already tried to do to solve your problem. This tells people you've already put in some work to figure things out on your own.
- List the version of Python you're using. Also, say which operating system and version you're running.
- If the error came up after you made a change to your code, explain exactly what you changed.
- Informe se você consegue reproduzir o erro sempre que executa o programa ou se ele ocorre somente após a realização de determinadas ações. Neste último caso, explique também quais são essas ações.

Sempre siga também as boas práticas de etiqueta online. Por exemplo, não escreva suas perguntas em letras maiúsculas nem faça exigências descabidas às pessoas que estão tentando te ajudar.

Você pode encontrar (https://autbor.com/help) mais informações sobre como pedir ajuda com programação na postagem do blog em ***https://autbor.com/help (https://autbor.com/help). (https://autbor.com/help)*** Adoro ajudar pessoas a descobrir o Python. Escrevo tutoriais de programação no meu blog em ***https://inventwithpython.com/blog (https://reddit.com/r/inventwithpython)e (https://inventwithpython.com/blog)*** você pode entrar em contato comigo com perguntas pelo e- *mail al@inventwithpython.com* ***, (https://autbor.com/help)embora (https://inventwithpython.com/blog)possa (https://inventwithpython.com/blog)obter (https://inventwithpython.com/blog)*** uma resposta mais rápida se postar suas ***perguntas (https://reddit.com/r/inventwithpython)em (https://reddit.com/r/inventwithpython)https://reddit.com/r/inventwithpython . (https://reddit.com/r/inventwithpython)***

Novidades da terceira edição

Esta edição totalmente revisada e atualizada inclui 16 novos projetos de programação para que você possa praticar as habilidades que aprenderá. Você encontrará uma introdução completa ao SQLite e a bancos de dados relacionais com o módulo sqlite3 do Python. Você explorará como compilar scripts Python em programas executáveis no Windows, macOS e Linux; e como criar e executar programas de linha de comando. Você também aprenderá como realizar operações com PDFs usando **PyPDF (https://pypdf.readthedocs.io/en/latest/)** e PdfMiner, usar o Playwright em conjunto com o Selenium para controlar navegadores da web e fazer seus programas se comunicarem com bibliotecas de conversão de texto em fala.Utilize a biblioteca Whisper da OpenAI para criar transcrições de texto a partir de arquivos de áudio e vídeo, extrair texto de imagens com PyTesseract, criar gráficos com matplotlib e muito mais.

Resumo

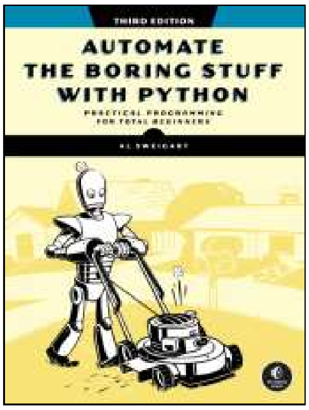
Para a maioria das pessoas, o computador é apenas um eletrodoméstico, e não uma ferramenta. Mas, ao aprender a programar, você terá acesso a uma das ferramentas mais poderosas do mundo moderno e se divertirá no processo. Programar não é nenhum bicho de sete cabeças — é perfeitamente normal que amadores experimentem e cometam erros.

Este livro parte do pressuposto de que você não possui nenhum conhecimento prévio de programação e lhe ensinará bastante, mas você poderá ter dúvidas que vão além do seu escopo. Lembre-se de que fazer perguntas pertinentes e saber como encontrar as respostas são ferramentas valiosas em sua jornada na programação.

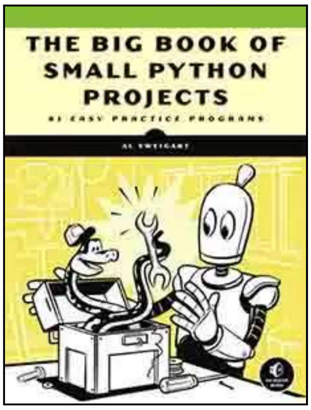
Vamos começar!

(fechar)

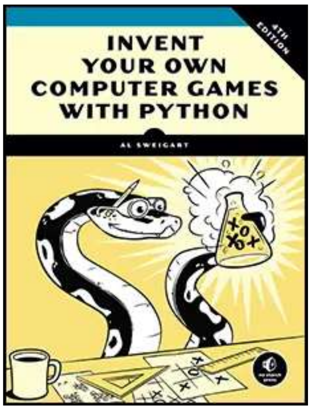
Confira outros livros de Al Sweigart, disponíveis **gratuitamente online (https://inventwithpython.com)** ou para compra: **...e outros livros também! (https://inventwithpython.com/)** Ou inscreva-se no **curso online em vídeo. (https://inventwithpython.com/automateudemy)** Você também pode **fazer uma doação para apoiar o autor diretamente. (https://inventwithpython.com/#donate)**

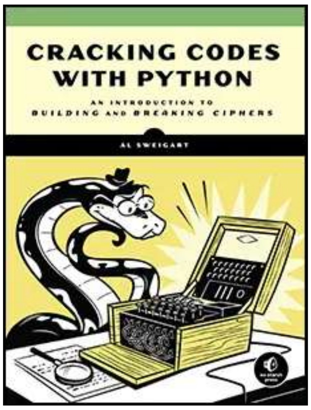


(https://nostarch.com/automate-boring-stuff-python-3rd-edition)



(https://nostarch.com/big-book-small-python-projects)





(https://nostarch.com/inventwithpython)
(https://nostarch.com/crackingcodes)
(https://inventwithpython.com/) (https://inventwithpython.com/automateudemy)
(https://inventwithpython.com/#donate)